

Architecture & Data Flow Diagrams

1. 層次架構圖（模組 / 依賴）

```
graph TD
    subgraph UI ["UI 層"]
        GUI["demo_gui_tk.py\n(Tkinter GUI)"]
    end

    subgraph Task ["任務層 (事件驅動)"]
        TP["task_parser_node\n規則式指令解析\n中文 / 英文"]
        PL["planner_node\n步驟規劃\n1s / step timer"]
    end

    subgraph Perception ["感知層 (15 Hz)"]
        CAM["camera_node\nUSB /dev/video0\n640×480 RGB"]
        PERC["perception_node\nONNX Runtime CPU\ndetection_v2.onnx\n224×224 CNN"]
    end

    subgraph Policy ["策略層 (50 Hz)"]
        POL["policy_node\nsimple_policy.onnx\nMLP (10→6)\nTwist action"]
    end

    subgraph HAL ["HAL 層 (1000 Hz)"]
        HB["hal_buddy_node\nIMU + MotorState\n+ Health"]
        H0["hal_omni_node\n(文件定義, \n尚未實作)"]
    end

    subgraph RT ["即時運控層 (500 Hz)"]
        SE["state_estimator\nIMU 積分姿態估算\nPose + TF"]
    end

    subgraph Bridge ["執行橋接層"]
        RB["robot_bridge_node\n步驟→馬達映射\n\nshoulder/elbow/\nwrist/gripper"]
    end

    subgraph Logging ["日誌層 (1 Hz status)"]
        REC["recorder_node\nMCAP bag writer\n/tmp/ros2bag"]
    end

    subgraph Models ["模型倉庫"]
        M1["simple_policy.onnx\nv0.1 Locomotion"]
        M2["detection.onnx\nv0.1 Detection"]
    end
```

```

    M3["detection_v2.onnx\n(active)"]
end

subgraph Deploy["部署層"]
    SYS["systemd\nrobot-core.service"]
    DC["Docker Compose\ndocker-compose.yml"]
    K3S["k3s\ndeploy-hal.yaml\ndeploy-policy.yaml"]
end

GUI -->|"/ui/user_command String"| TP
TP -->|"/task/parsed_command JSON"| PL
PL -->|"/planner/current_step Int32"| RB

CAM -->|"/camera/image_raw Image"| PERC
PERC --> M3

HB -->|"/buddy/imu Imu 1kHz"| SE
HB -->|"/buddy/imu Imu 1kHz"| POL
SE -->|"/state/pose Pose"| REC
SE -->|"/tf TransformStamped"| REC

POL --> M1
POL -->|"/policy/action Twist"| REC

REC -->|"/recorder/status"| GUI

SYS -. ->| "ExecStart" | HB
DC -. ->| "service" | HB
K3S -. ->| "deploy" | HB
K3S -. ->| "deploy" | POL

M2 -. ->| "fallback" | PERC

style UI fill:#dbeafe,stroke:#3b82f6
style Task fill:#fef9c3,stroke:#ca8a04
style Perception fill:#dcfce7,stroke:#16a34a
style Policy fill:#fcef7f,stroke:#db2777
style HAL fill:#fee2e2,stroke:#dc2626
style RT fill:#ffedd5,stroke:#ea580c
style Bridge fill:#ede9fe,stroke:#7c3aed
style Logging fill:#f0fdf4,stroke:#15803d
style Models fill:#f1f5f9,stroke:#64748b
style Deploy fill:#e0f2fe,stroke:#0284c7

```

2. 資料流圖 (ROS 2 Topics / IPC 邊界)

flowchart LR

subgraph ext["外部邊界"]

USB["USB Camera\n/dev/video0"]

HW["硬體馬達\n(sim=True\n目前模擬)"]

USER["使用者輸入\nGUI / CLI"]

end

subgraph ros2["ROS 2 DDS 匯流排 (ROS_DOMAIN_ID=42 / CycloneDDS)"]

direction TB

T1["/ui/user_command\nString"]

T2["/task/parsed_command\nString (JSON)"]

T3["/planner/task_plan\nFloat32MultiArray"]

T4["/planner/current_step\nInt32"]

T5["/robot/state\nString"]

T6["/robot/motor_status\nFloat32MultiArray"]

T7["/buddy/imu\nsensor_msgs/Imu\n1000 Hz"]

T8["/buddy/motor_state\nTwist 1000 Hz"]

T9["/buddy/hal/health\nString 1000 Hz"]

T10["/state/pose\nPose 500 Hz"]

T11["/tf\nTransformStamped\n500 Hz"]

T12["/camera/image_raw\nImage 15 Hz"]

T13["/perception/detections\nDetection2DArray"]

T14["/perception/latency\nFloat32MultiArray"]

T15["/policy/action\nTwist 50 Hz"]

T16["/policy/action_chunk\nFloat32MultiArray\n50 Hz"]

T17["/policy/latency\nFloat32MultiArray\n1 Hz"]

T18["/recorder/status\nString 1 Hz"]

end

subgraph nodes["ROS 2 Nodes"]

N_TP["task_parser_node"]

N_PL["planner_node"]

N_RB["robot_bridge_node"]

N_HB["hal_buddy_node"]

N_SE["state_estimator"]

N_CAM["camera_node"]

N_PERC["perception_node"]

N_POL["policy_node"]

N_REC["recorder_node"]

end

```
subgraph inference["推理邊界 (ONNX Runtime)"]
    ORT_D["detection_v2.onnx\nCPU Session"]
    ORT_P["simple_policy.onnx\n(待整合)"]
end
```

```
subgraph storage["存儲邊界"]
    BAG["/tmp/ros2bag\nMCAP bag"]
    LOG["$POC_ROOT/logs/\nSystem logs"]
end
```

```
USER -->|"GUI click / text"| T1
T1 --> N_TP
N_TP -->|"JSON parse"| T2
T2 --> N_PL
N_PL --> T3
N_PL --> T4
T4 --> N_RB
N_RB --> T5
N_RB --> T6
T5 -->|"state feedback"| USER
T6 --> HW
```

```
USB -->|"v4l2 frame"| N_CAM
N_CAM --> T12
T12 --> N_PERC
N_PERC -->|"run()"| ORT_D
ORT_D -->|"bbox + conf"| N_PERC
N_PERC --> T13
N_PERC --> T14
```

```
N_HB --> T7
N_HB --> T8
N_HB --> T9
T7 --> N_SE
T7 --> N_POL
N_SE --> T10
N_SE --> T11
N_POL -->|"run() (待整合)"| ORT_P
N_POL --> T15
N_POL --> T16
N_POL --> T17
```

```
T7 --> N_REC
T10 --> N_REC
T15 --> N_REC
```

```
N_REC --> T18
N_REC -->|"write"| BAG
T18 -->|"status"| USER
```

```
style ext fill:#fef3c7,stroke:#d97706
style ros2 fill:#f0f9ff,stroke:#0369a1
style nodes fill:#f5f3ff,stroke:#7c3aed
style inference fill:#fdf2f8,stroke:#be185d
style storage fill:#f0fdf4,stroke:#15803d
```

3. Bring-up 啟動序列

```
sequenceDiagram
    participant SYS as systemd / 操作員
    participant CORE as bringup_core.sh
    participant CTRL as bringup_control.sh
    participant PERC as bringup_perception.sh
    participant HAL as hal_buddy_node
    participant SE as state_estimator
    participant POL as policy_node
    participant CAM as camera_node
    participant PERC_N as perception_node
    participant REC as recorder_node

    SYS->>CORE: bringup_all.sh
    CORE->>CORE: check ROS 2 Humble
    CORE->>CORE: source setup.bash
    CORE->>CORE: mkdir logs/
    CORE-->>SYS: Core OK

    SYS->>CTRL: bringup_control.sh
    CTRL->>HAL: ros2 run hal_buddy_node (1kHz)
    HAL-->>CTRL: /buddy/imu, /buddy/motor_state, /buddy/hal/health
    CTRL->>SE: ros2 run state_estimator (500Hz)
    SE-->>CTRL: /state/pose, /tf
    CTRL->>POL: ros2 run policy_node (50Hz)
    POL-->>CTRL: /policy/action, /policy/action_chunk
    CTRL->>REC: ros2 run recorder_node
    REC-->>CTRL: /recorder/status
    CTRL-->>SYS: Control OK

    SYS->>PERC: bringup_perception.sh
    PERC->>CAM: ros2 run camera_node (15Hz)
    CAM-->>PERC: /camera/image_raw
    PERC->>PERC_N: ros2 run perception_node
```

Note over PERC_N: Load detection_v2.onnx
via ONNX Runtime CPU
PERC_N-->>PERC: /perception/detections
PERC-->>SYS: Perception OK

SYS-->>SYS: System Ready → launch_demo.sh / launch_mission.sh

Topic 速查表

Topic	Type	Hz	Publisher → Subscriber
/ui/user_command	String	event	GUI → task_parser
/task/parsed_command	String (JSON)	event	task_parser → planner
/planner/current_step	Int32	1/s	planner → robot_bridge
/robot/state	String	event	robot_bridge → GUI
/robot/motor_status	Float32MultiArray	event	robot_bridge → HW
/buddy/imu	sensor_msgs/Imu	1000	hal_buddy → state_estimator, policy, recorder
/buddy/motor_state	Twist	1000	hal_buddy
/buddy/hal/health	String	1000	hal_buddy
/state/pose	Pose	500	state_estimator → recorder
/tf	TransformStamped	500	state_estimator
/camera/image_raw	Image	15	camera_node → perception_node
/perception/detections	Detection2DArray	15	perception_node
/perception/latency	Float32MultiArray	15	perception_node
/policy/action	Twist	50	policy_node → recorder
/policy/action_chunk	Float32MultiArray	50	policy_node
/policy/latency	Float32MultiArray	1	policy_node
/recorder/status	String	1	recorder_node → GUI